

ПЛАН ДЛЯ FRONTEND-КОМАНДЫ

# Выбор LLM-провайдера и модели

UI-контракт для React-приложения после подключения FastAPI

## Главный результат

Пользователь выбирает провайдера, получает список доступных моделей, проверяет соединение и сохраняет глобальную настройку без прямых запросов frontend к внешним LLM.

Версия 1.0 · 6 сентября 2026

# 1. Границы ответственности

Frontend отвечает только за представление и вызов локального FastAPI. Правила доступности, URL, авторизации, сертификата и поддерживаемых моделей приходят с backend.

## Запрещено

Прямые запросы к GigaChat, OpenAI, Anthropic, Ollama, vLLM или custom router; хранение ключа в localStorage/sessionStorage; собственная проверка сертификата; дублирование backend-валидации.

## Пользовательские провайдеры

| Provider ID       | Название          | Поля                                            |
|-------------------|-------------------|-------------------------------------------------|
| gigachat          | GigaChat          | Ключ, модель, статус сертификата                |
| openai            | OpenAI            | API-ключ, модель                                |
| anthropic         | Anthropic         | API-ключ, модель                                |
| openai_compatible | OpenAI-compatible | HTTPS/loopback URL, необязательный ключ, модель |
| ollama            | Ollama            | Loopback URL, модель                            |
| vllm              | vLLM              | Loopback/HTTPS URL, необязательный ключ, модель |

- Показывать все шесть карточек; mock отображать только в development-сборке.
- Provider с available=false показывать отключённым вместе с disabled\_reason.
- Сетевая ошибка не скрывает provider и не стирает заполненную форму.

## 2. API-контракт и состояние формы

Названия маршрутов являются контрактом с backend-командой. Frontend формирует русские подписи, но машинные коды не переименовывает.

### Запросы

| Метод                     | Назначение                                            | Правило UI                                    |
|---------------------------|-------------------------------------------------------|-----------------------------------------------|
| GET /api/config           | Активная конфигурация, has_api_key и каталог provider | Начальная загрузка экрана                     |
| POST /api/config/models   | Список моделей по черновой конфигурации               | Не сохраняет ключ и настройки                 |
| POST /api/config/validate | Синтетическая проверка выбранной модели               | Может использовать минимальную облачную квоту |
| PUT /api/config           | Сохранение глобального выбора                         | Ключ write-only; ответ его не содержит        |

### Локальное состояние

- Хранить provider, model, base\_url, временное значение ключа и состояния запросов только в React state.
- Пустое поле ключа означает «оставить сохранённый ключ»; удаление выполняется отдельным явным действием.
- После успешного PUT очистить ключ из state и DOM, затем использовать только has\_api\_key.
- При смене provider сбрасывать model list и provider-specific ошибки, но не отправлять запрос автоматически.

### Состояния запросов

| Состояние      | Поведение                                                     |
|----------------|---------------------------------------------------------------|
| idle           | Поля доступны, результат предыдущей проверки скрыт            |
| loading_models | Блокировать повторную загрузку моделей, сохранить ручной ввод |
| validating     | Блокировать validate/save и показать локальный progress       |
| saving         | Блокировать форму; после успеха очистить ключ                 |

### 3. Сценарий выбора модели

#### Основной поток

| Шаг | Действие                            | Результат                                       |
|-----|-------------------------------------|-------------------------------------------------|
| 1   | Выбрать карточку provider           | Показать только применимые поля                 |
| 2   | Ввести ключ/URL при необходимости   | Локальная проверка обязательности полей         |
| 3   | Нажать «Загрузить модели»           | Заполнить searchable combobox                   |
| 4   | Выбрать модель или ввести точный ID | Ручной ID отмечается как непроверенный          |
| 5   | Нажать «Проверить подключение»      | Показать результат без технических деталей      |
| 6   | Нажать «Сохранить»                  | Обновить публичную конфигурацию и очистить ключ |

#### Combobox моделей

- Поддерживать поиск, клавиатуру, loading, пустой список, повторную загрузку и ручной ввод.
- Показывать display\_name, но сохранять неизменный id.
- При недоступном model list не блокировать ручной ввод; обязательной остаётся validate-проверка.

#### Ошибки

| Код                          | Пользовательский текст                                       |
|------------------------------|--------------------------------------------------------------|
| missing_credentials          | Введите ключ для выбранного провайдера                       |
| invalid_base_url             | Проверьте адрес сервера. HTTP разрешён только для localhost  |
| authentication_failed        | Не удалось подтвердить ключ авторизации                      |
| model_not_found              | Выбранная модель недоступна                                  |
| unsupported_model_capability | Модель не поддерживает требуемый структурированный ответ     |
| connection_failed            | Не удалось подключиться. Проверьте сеть или локальный сервер |

## 4. Безопасность и приёмка

### Обязательные проверки frontend

- API-ключ отсутствует в localStorage, sessionStorage, URL, логгах, telemetry и snapshot-тестах.
- После save, закрытия модального окна и смены provider значение ключа удаляется из DOM.
- Все production-запросы используют относительный /api; внешних fetch/axios-вызовов нет.
- Недоступный provider виден, отключён и содержит безопасную причину.
- Другие provider остаются работоспособными при ошибке сертификата GigaChat.
- Тесты покрывают переключение карточек, dynamic fields, combobox, ручной ID и состояния запросов.

### Критерии готовности

#### Функциональная приёмка

Пользователь выбирает каждый из шести provider, получает или вручную вводит модель, проверяет соединение и сохраняет глобальную настройку. Текущий ключ никогда не возвращается с backend.

#### Техническая приёмка

Vitest, typecheck и production build проходят; поиск по frontend не находит прямых доменов LLM, verify=false, сохранения ключа или внешней аналитики.

### Зависимости команды

- Начать интеграцию после того, как backend-команда отдаст финальные DTO и маршруты config/models/validate.
- Не реализовывать обходной mock API и не переносить backend-правила во frontend.
- Согласовать только русские тексты и визуальное представление; машинные значения стабильны.

Этот документ является дополнением к основному frontend-плану и касается только выбора LLM.